

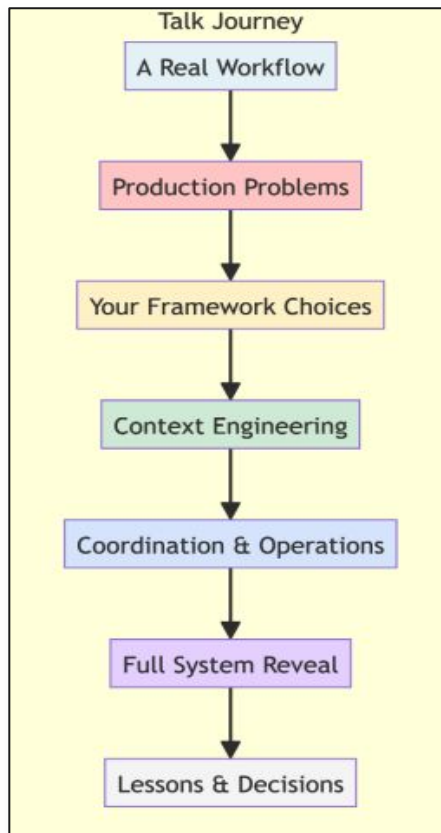


Agentic AI in Prod: Architectures, Memory, Deployment & Reliability

Raunak Bhandari | Ex-Google Gemini / Search

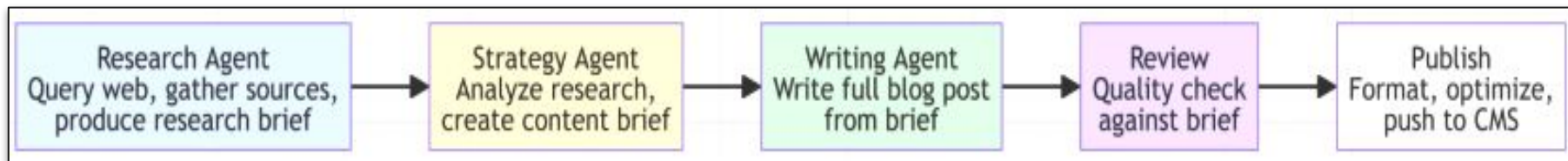
Agentic AI in Prod: Architectures, Memory, Deployment & Reliability

- Raunak Bhandari — Ex-Google (Gemini / Search)
- What?
 - What actually broke when we shipped agents to production
 - Solution patterns for each problem
- Takeaways
 - concrete, transferable patterns for reliable AI agents
 - a decision framework for YOUR system



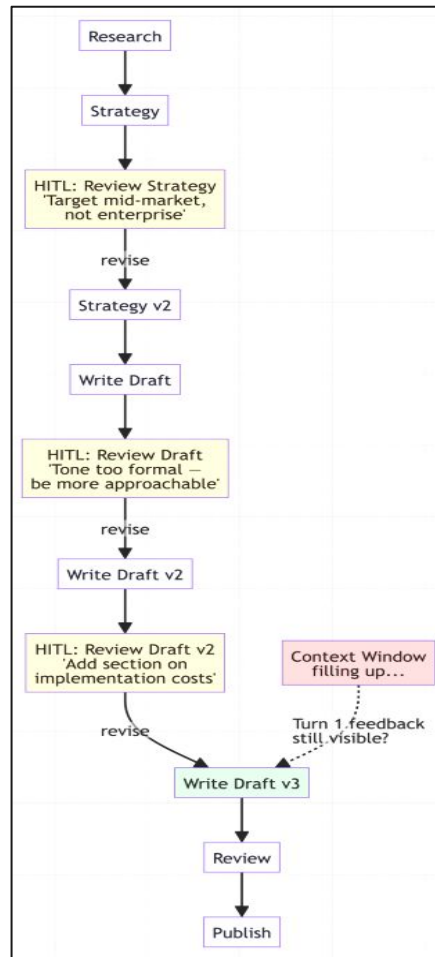
A Real Workflow: The Happy Path

- A real marketing content creation workflow — 5 agents, linear pipeline
- Research → Strategy → Content Write → Review → Publish
- Single user, happy path, 3-5 minutes end to end
- This is the version that works in every demo. The audience applauds.
- Now let's add reality.



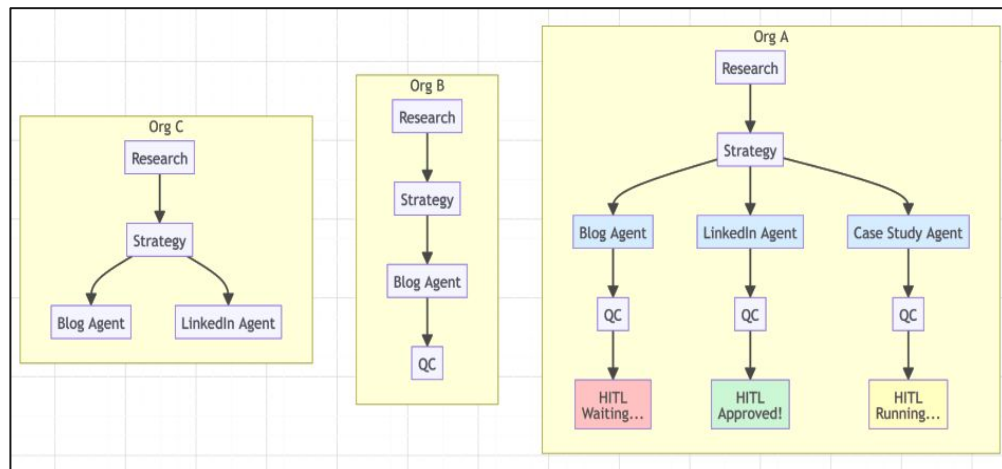
Add Real Users: HITL + Multi-Turn

- Same workflow, but now a human gives feedback, agent revises
 - multiple rounds
 - Turn 1: "Target mid-market, not enterprise"
 - Turn 3: "Tone too formal"
 - Turn 5: "Add cost section"
- Workflow stretches from 2 minutes to 30-60 minutes with human review gaps
- By turn 5, the agent needs to remember what the user said in turn 1
 - but context is overflowing
- The agent "forgets" brand voice feedback
 - not because it's dumb, but because old messages got pushed out



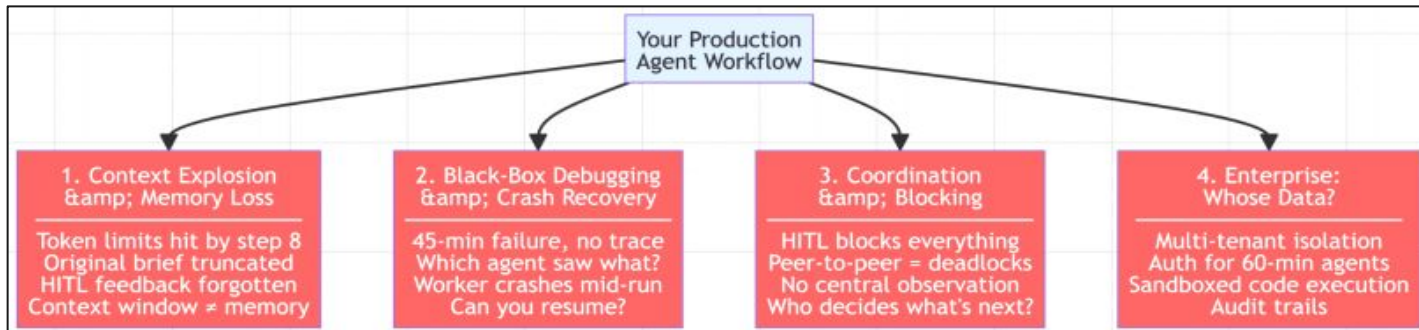
Add Scale + Parallel Work

- 3 content pieces in parallel
- Add enterprise customers
 - 3 orgs on shared infrastructure, each with their own data
- Problems emerge:
 - **Data isolation:** Org A's data must never leak to Org B
 - **Orchestration:** who decides what's next?
 - **Observability:** 9 parallel workflows — something fails?
 - **Context Engineering:** Agent must have context!

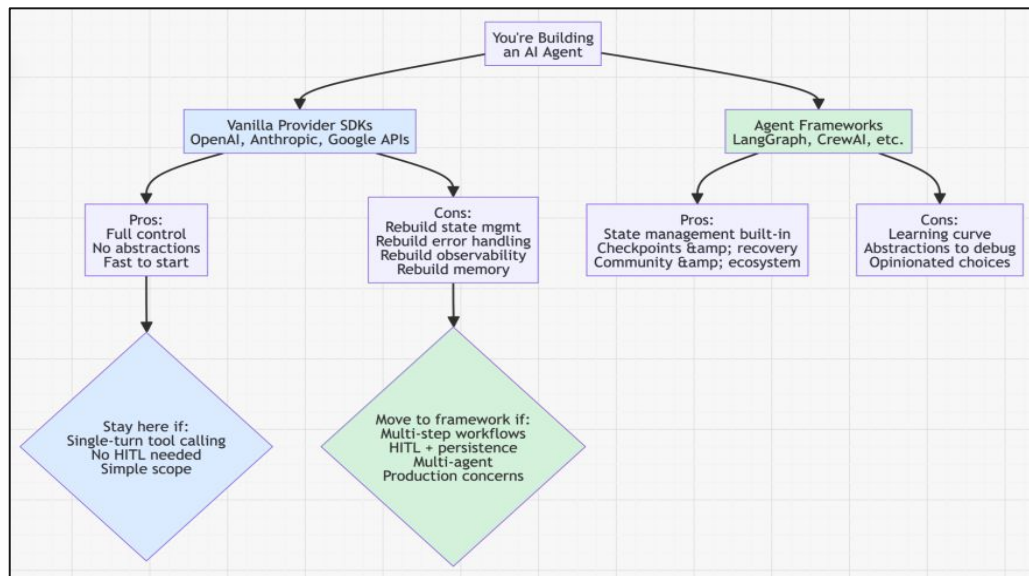


The Four Production Problems

- **Context Explosion & Memory Loss** — token limits hit by step 8, original brief truncated, HITL feedback forgotten. Context window is NOT memory.
- **Observability, Black-Box Debugging & Crash Recovery** — 45-min failure with no trace. Worker crashes mid-run — can you resume?
- **Orchestration** — HITL blocks everything. Peer-to-peer = deadlocks. No central observation point.
- **Enterprise: Whose Data?** — multi-tenant isolation, auth for 60-min agents, sandboxed execution, audit trails.
- "The rest of this talk: how we solved each one."



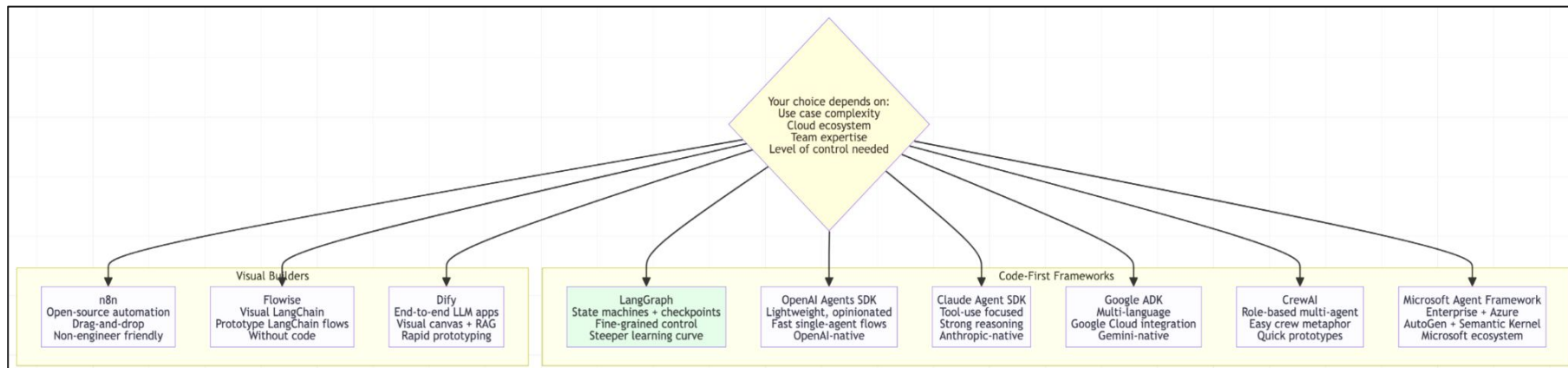
Starting Point: Your Agent Development Choices



- The fork every team faces: **Vanilla Provider SDKs** (OpenAI, Anthropic, Google) vs. **Agent Frameworks**
- "Most teams underestimate the production gap."

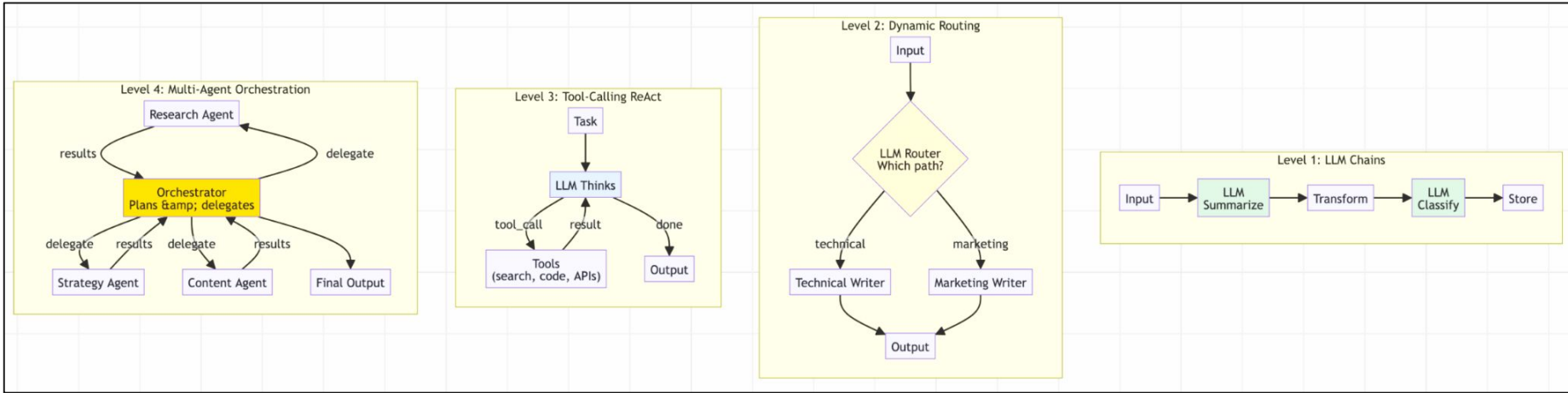
Framework Landscape (Quick Guide)

- Maintain fluidity! Frameworks are evolving super fast.
- There's no such thing as the best framework, these are all guesses by super-smart people. But guesses.
- Fast POCs >> Framework Lock-In

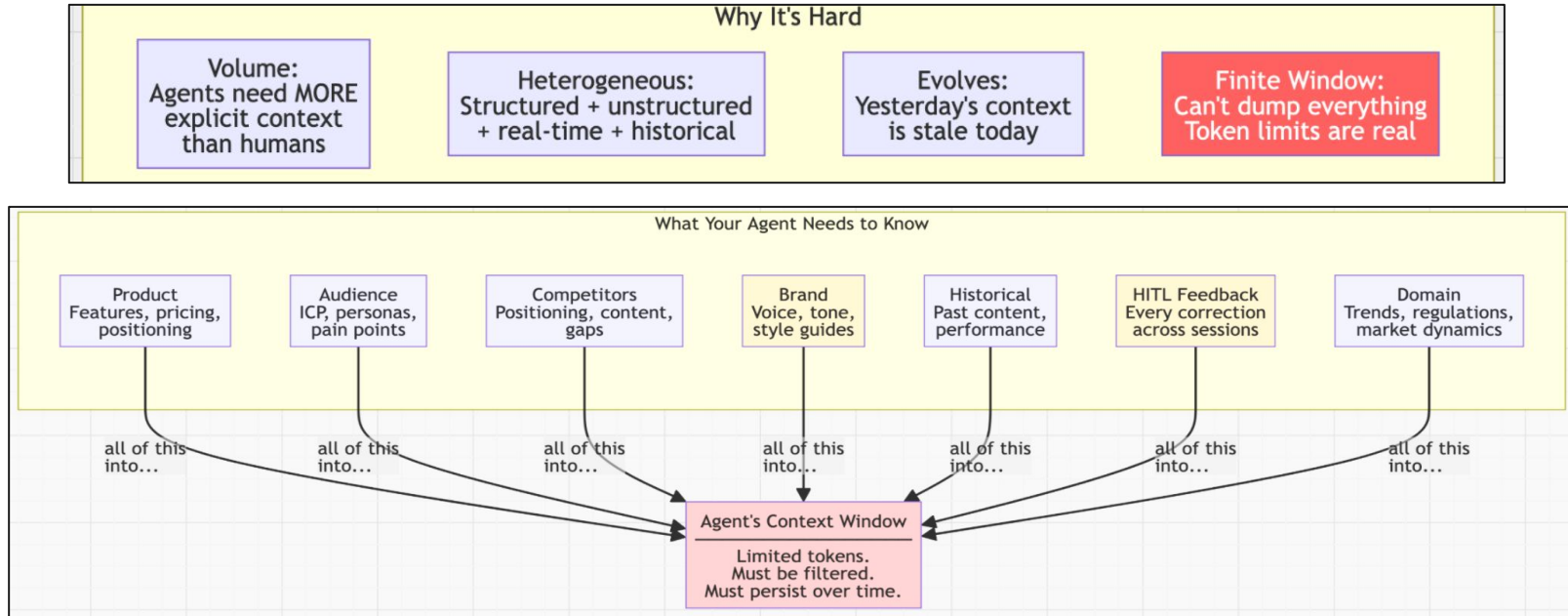


The AI Agent Autonomy Spectrum

- Tradeoff: More Degrees of Freedom == More powerful == More chances to fail

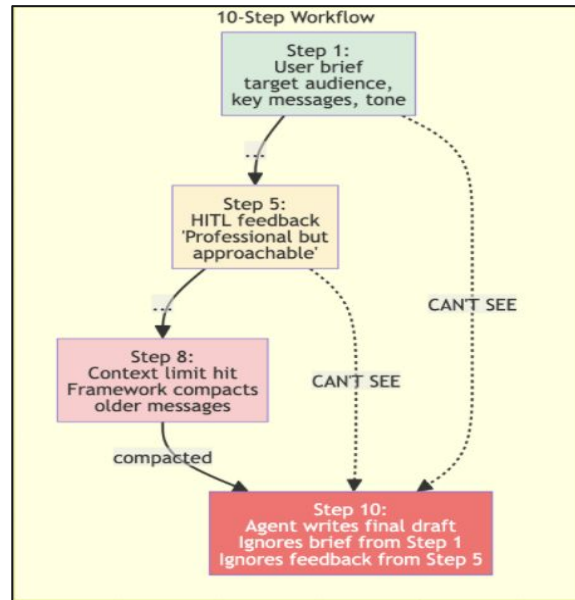
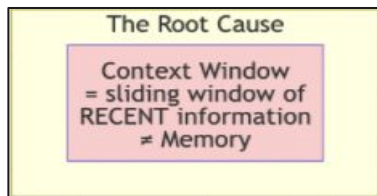


Context Engineering Is THE Production Challenge



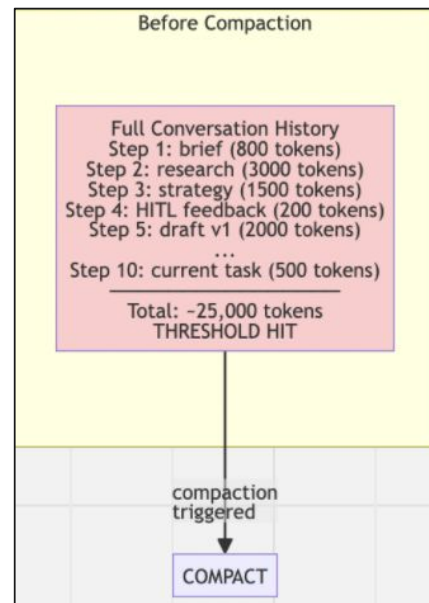
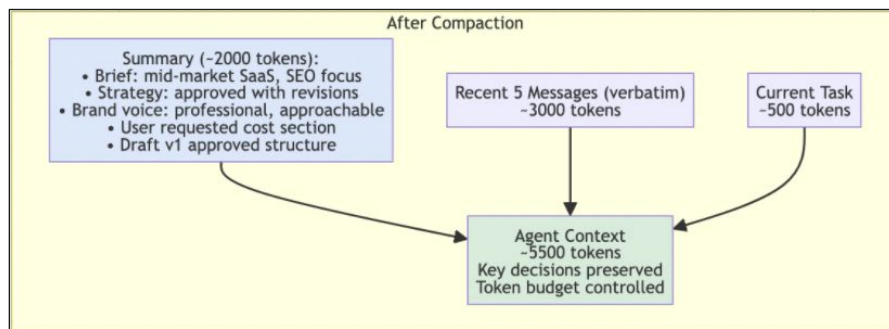
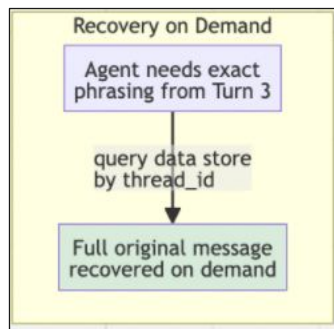
Your Agent will Forget

- Context window saturation or Prompt Compaction (lossy compression)
- The catastrophic forgetting pattern:
 - Step 1: User provides detailed brief (target audience, tone, key messages)
 - Step 5: User gives feedback — "professional but approachable"
 - Step 8: Token limit hit — framework compacts older messages
 - Step 10: Agent writes final draft **ignoring brief and feedback** — they were compacted away
- **Context window is NOT memory in long chats** — it's a sliding window of recent information. Anything outside is gone.



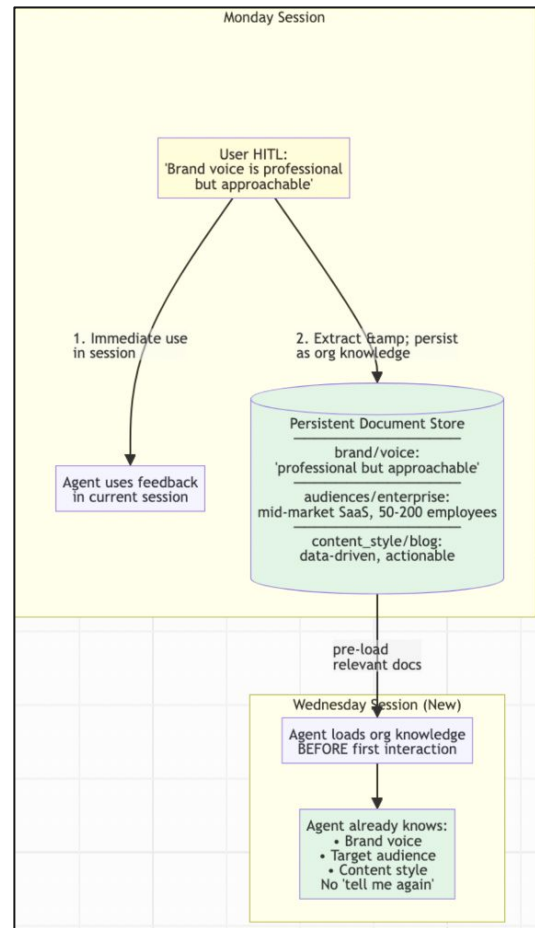
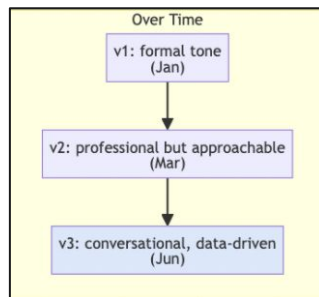
Solution #1: Session Memory — Smart Compaction

- **Compaction as ML problem:** evaluate!
 - Agents are non-deterministic ML systems and you must eval every component!
- All data that flows in to agent must be stored and queryable!
 - post compaction, Agent fetches missing context on-demand
- **Solves:** context window overflow within a single session
- **Stop here if:** short-lived agents, single session, no cross-session needs



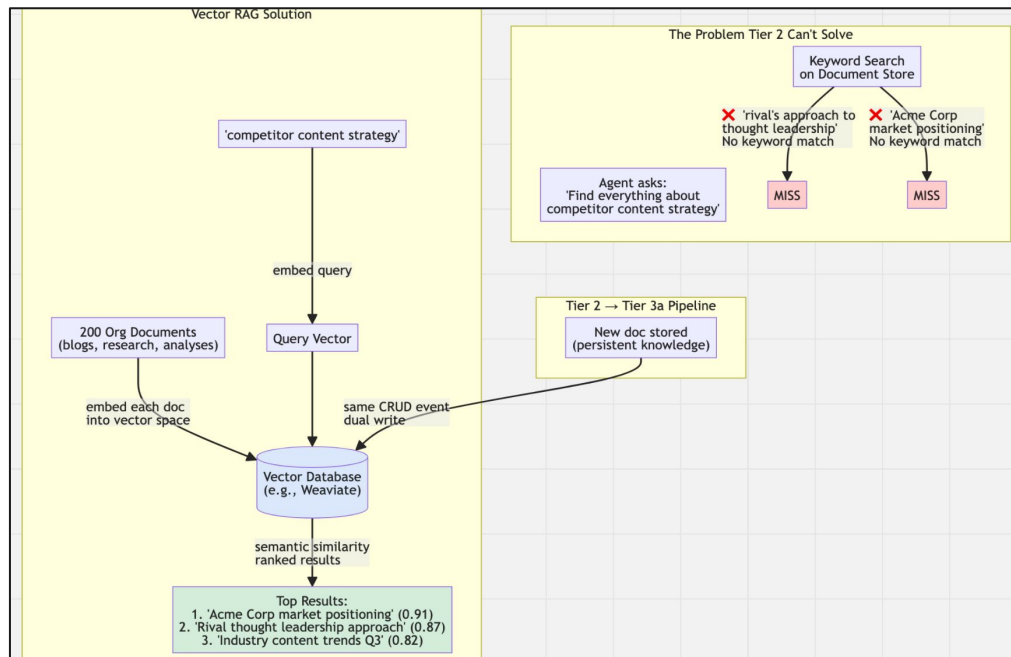
Sol. #2: Long-Term Memory — Persistent Knowledge

- Build a repo of customer docs, just like Claude Code
- Ingest customer documents in your system for context
- Capture HITL feedback as **organizational knowledge**
- Store with logical namespaces: `brand/voice`, `audiences/enterprise`, `content\_style/blog`
- Future sessions start **pre-loaded** with org knowledge
- **Provide tools for agentic search**
- Document RAG sometimes works better than vector RAG
 - (no retrieval tuning, full context)

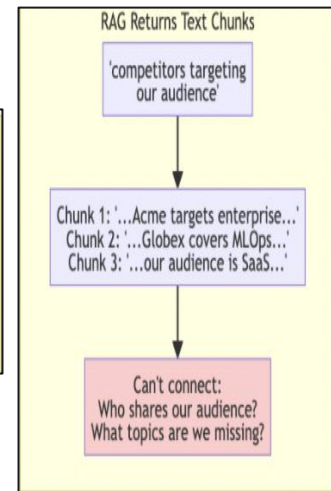
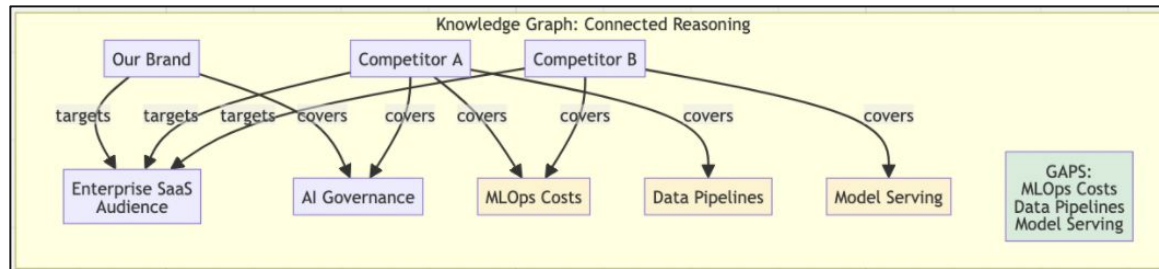
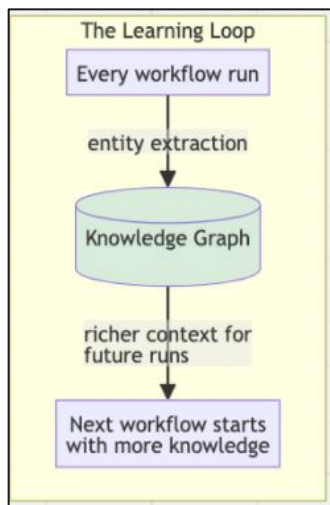


Solution #3: Vector RAG

- If Agent needs to search == Vector RAG
- more efficient chunks than loading full docs
- **Tradeoff:** full docs with agentic search or vector chunks!
- Doesn't solve: relationships in data

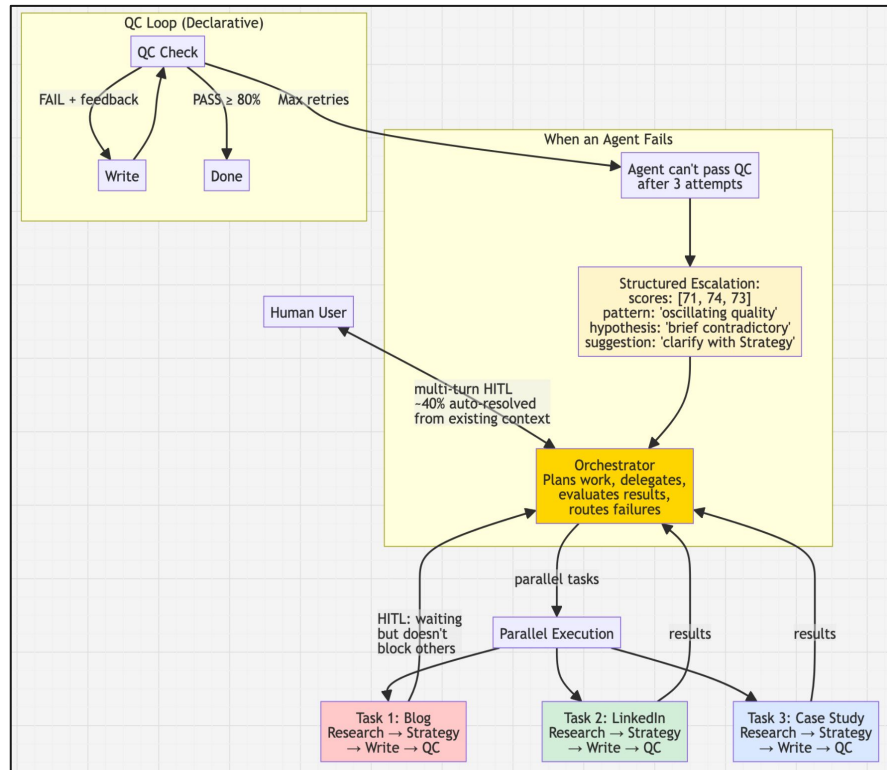


Solution #4: Knowledge Graph

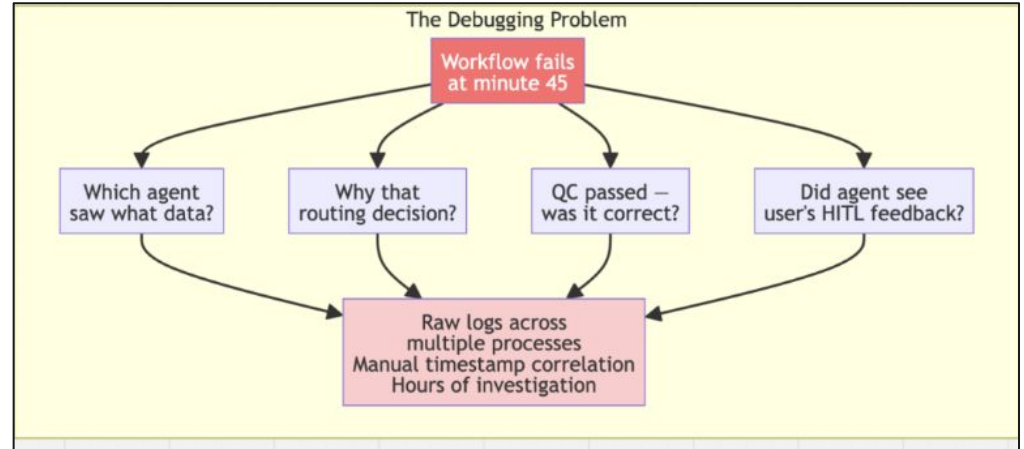
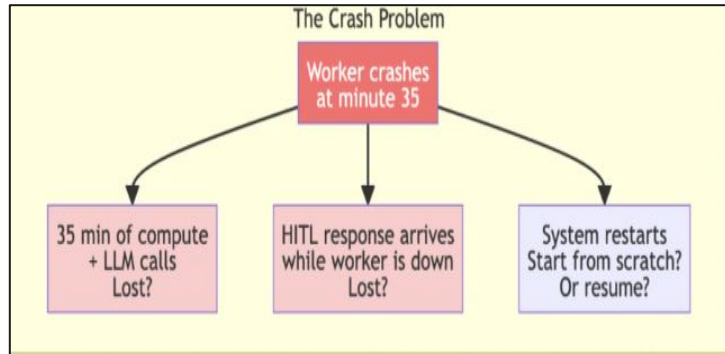


Pattern: Multi-Agent Architectures

- 90% of you don't need this
- Hard to debug, eval, improve
- But if you do...
- Try: **hub-and-spoke model**: central orchestrator agent
 - All agents communicate ONLY through the orchestrator.
 - Never direct agent-to-agent calls

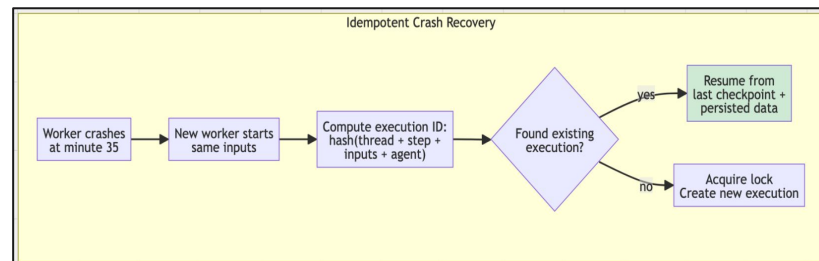
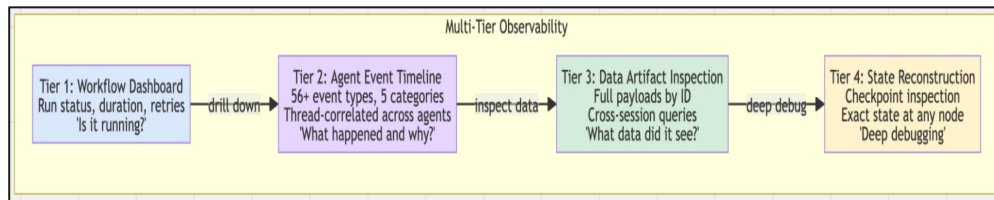
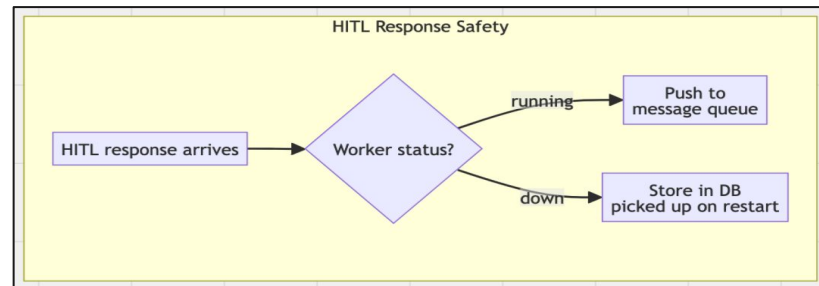


Problem: Black-Box Debugging + Crashes



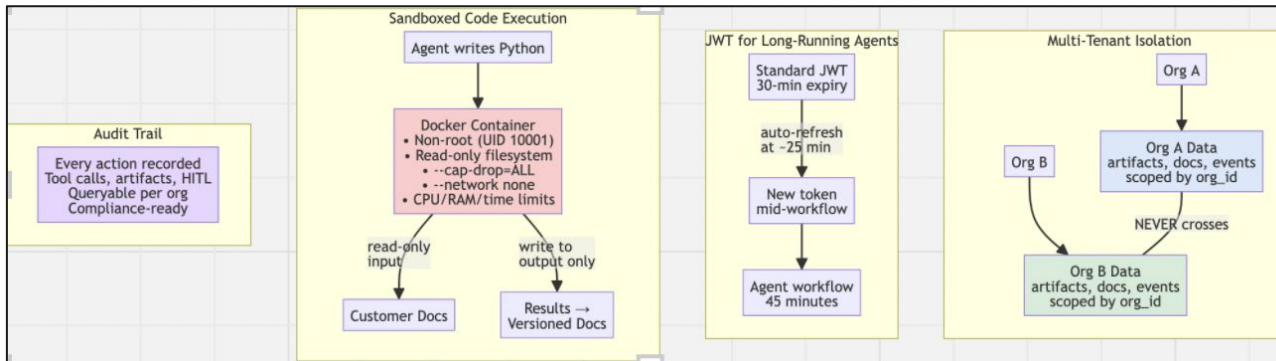
Solution: Multi-Tier Observability + Idempotent Recovery

- Always plan for failures
- Observability - progressively drill-down:
 - messages history
 - workflow dashboard (eg: Prefect)
 - Logging
 - inspectable agent handoffs, tool request / responses, HITL messages
 - Recommend: Langfuse (OSS) - agent traces

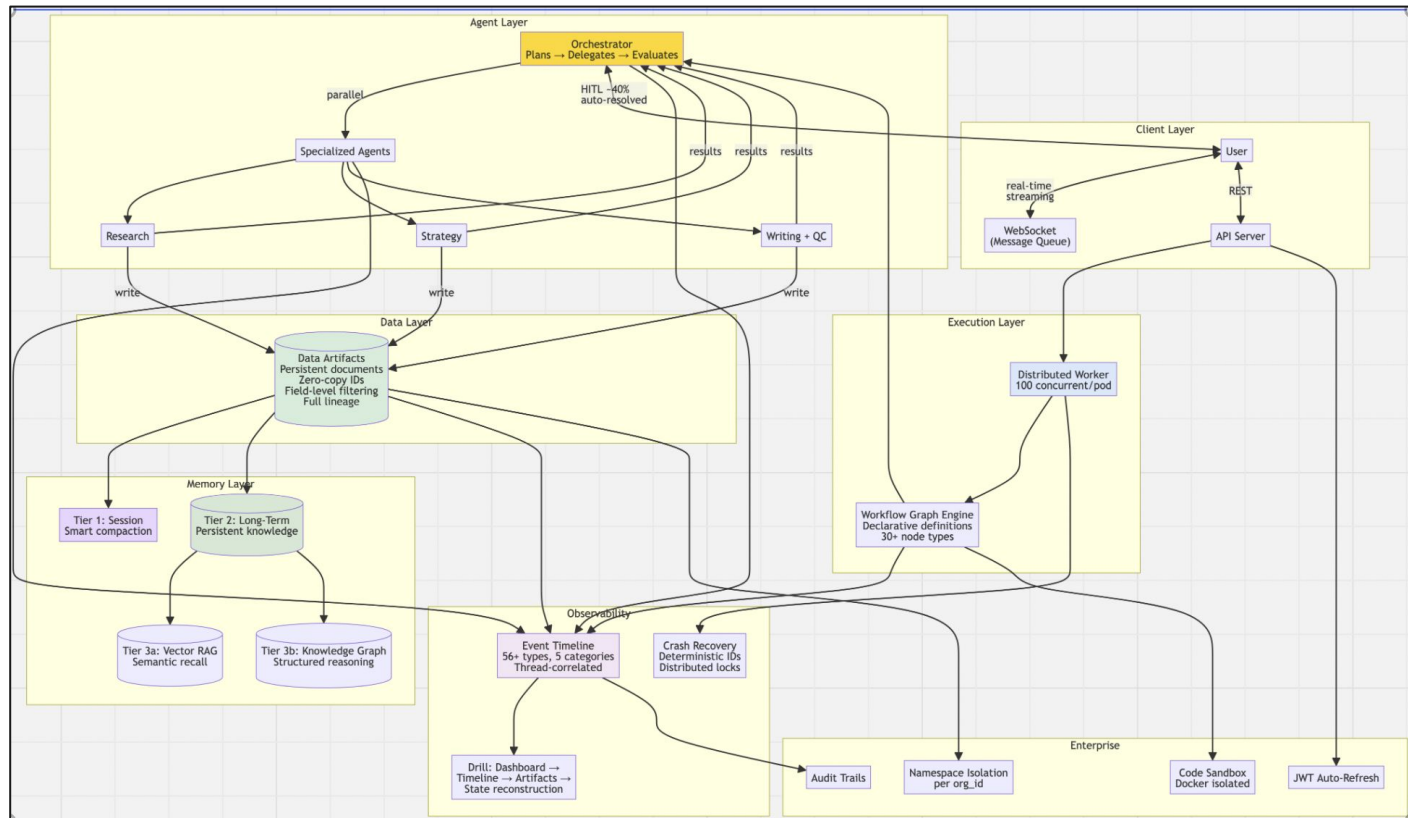


Enterprise: The Stuff Demos Never Show

- **Painful path to SOC II**
- **Multi-tenant isolation:** every artifact, document, event scoped by org ID — Org A's data never appears in Org B's context
- **JWT auto-refresh for long-running agents:** standard JWT expires in 30 min; agents run 10-60+ min — auto-renew credentials mid-workflow
- **Sandboxed code execution:** Docker container — non-root, read-only filesystem, all capabilities dropped, network isolated, CPU/RAM/time limits
- **Audit trails:** every tool call, artifact creation, HITL interaction recorded — queryable per org, compliance-ready



Full E2E Production System



Lessons Learned

5 Hard-Won Lessons:

- Agents are non-deterministic ML systems and you must eval every component!
- Context engineering > prompt engineering — perfect prompt + forgotten context = bad output
- Start with observability — build event instrumentation before features, not after the first incident
- Design for failure from day one — deterministic IDs, idempotent ops, auto-expiring locks.
- Choose boring, reduce degrees of freedom, build guardrails. Predictable, debuggable, testable.
 - NOTE: with current quality of frontier LLMs, this may no longer be true! But still need robust guardrails

1. Context & Prompts
Architecture beats engineering
'Perfect prompt, forgotten
context = bad output'

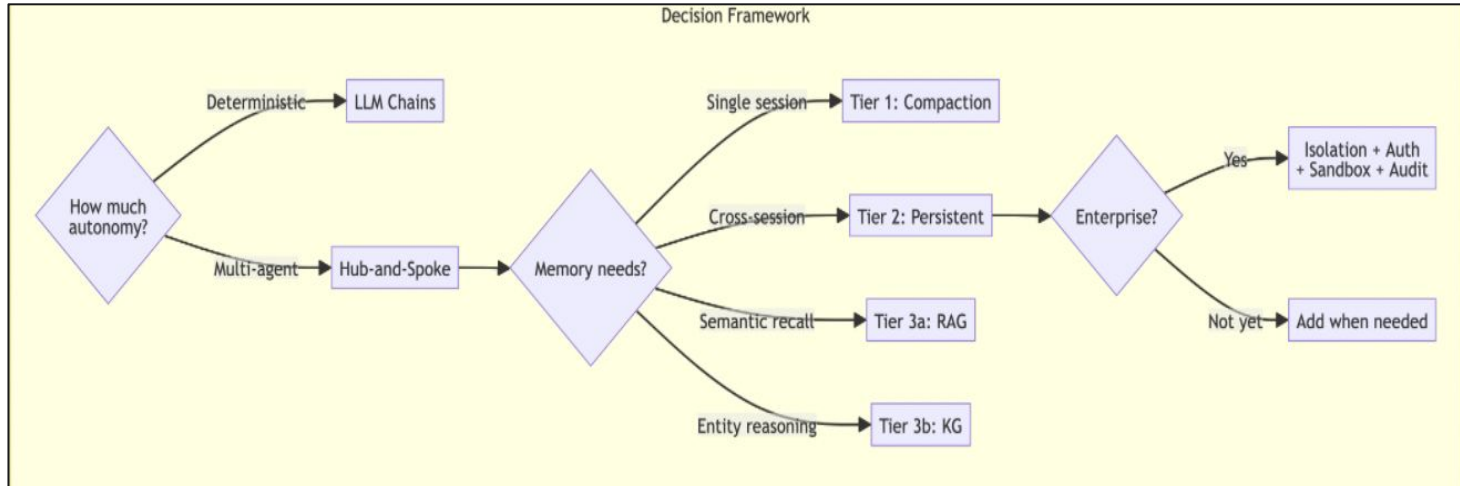
2. Observability FIRST
Before features
'If you can't see it,
you can't fix it'

3. Design for Failure
Before scaling
'Every long-running
op WILL fail'

4. Choose Boring
Hub-and-spoke, not peer-to-peer
'Predictable, debuggable,
testable'

5. Declarative Quality
Config flag, not custom code
'95% of QC patterns,
one flag'

Decision Framework





Q&A

Thank you!

Raunak Bhandari

[linkedin.com/in/raunakbhandari](https://www.linkedin.com/in/raunakbhandari)